

Class: 25

20/05/2025

Has-A relationship

* Two types.

1. Aggregation } called Associations.
2. Composition.

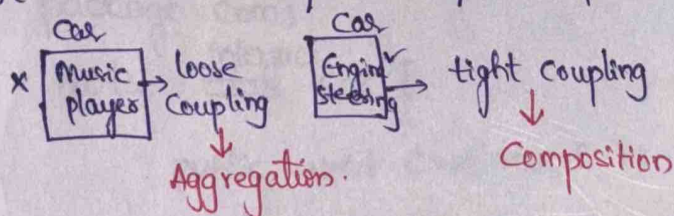
* It has one class having an object of another class.

Ex: ~~Class~~ ^{Car} ~~Engine~~ {
 ^{Class} Engine ^{reference variable} engine; (Car has a Engine)
 String ^{Class} name;
}

class Engine {
 String model;
 String capacity;

Aggregation:

* It is a very loosely coupled.



Composition:

* It is a tight coupling b/w parent & child class.

* When a parent class and child class tightly coupled then it is called as Composition.

* Parent doesn't make sense without the child class.

Aggregation:

* When parent & child class loosely coupled then it is called as Aggregation.

* Parent class make sense without child class.

ex

package com;

public class Professor {

String name;

String Subject;

int experience;

}

package com;

public class Department {

// Has-A relation

(Aggregation)

Professor professor;

String name;

int countOfStudents;

}

package com;

public class College {

(Composition)

String CollegeName;

Department department;

}

package com

public class Test {

PSUM {

Professor professor = new Professor();

professor.name = "Rani";

professor.experience = 10;

professor.subject = "PCB";

Department department = new Department();

department.name = "PCB";

department.CountOfStudents = 1000

department.setProfessor(new Professor("xxx", "Python", 20));

POJO class - Plain Old Java Objects.

↳ ~~Take~~ having - { private variables
getters & setters
to string methods
some constructors }

department.setProfessor(new Professor("xxx", "Python", 20));

output: PCB
1000
Ravi
10

System.out.println(department);

Ex: package com;
public class User {
 department.setProfessor(new Professor("xxx", "Python", 20));
}

WRAPPER CLASSES

- * byte → Byte class
- * int → Integer class
- * short → Short class
- * long → Long class
- * char → Character class
- * float → Float class
- * boolean → Boolean class.

wrapper classes.

int $\xrightarrow{\text{Auto Boxing}}$ Integer
Integer $\xleftarrow{\text{Auto Un Boxing}}$ int

- * Converting int variable to Integer - Auto Boxing
- * Converting Integer to int variable - Auto Un Boxing.

Ex: package com;
public class Test {
 public static void main {

int a = 10;

Integer b = new Integer(20); // Auto boxing.

Integer c = Integer.valueOf(a);
// Auto unboxing. Static method.

int d = b.intValue();

int f = b;

Integer e = a; (we can also write like this, JVM internally call this)
(After Java 5)

Output: 10
20

* Collections work with only class & Objects.

* Default methods:

Ex: String s = "200"; ~~;~~ sysout(s+2); Output: 2002.

int num = Integer.parseInt(s); → [Converts string to int value]

sysout(~~num~~ 2); Output: 202

String s1 = String.valueOf(num);

Exception Handling

*

Ex:

```

package com;

public class Calculator {

    public static void main (String[] args) {

        Scanner sc = new Scanner(System.in);
        Syso (" Enter numbers 1: ");
        int num1 = sc. nextInt ();
        syso (" Enter numbers 2: ");
        int num2 = sc. nextInt ();
        int num3 = num1/num2 ;
        Syso (" After division : " + num3);

    }

}

```

Output:

Enter number 1:

100

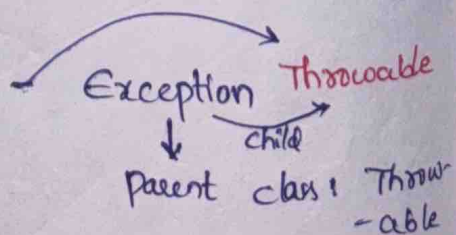
Enter number 2:

10

After Division: 10

Exceptions:

- * This causes a abnormal termination of your program
- * 2 types: (Ex: File Not Found Exception), SQL exce, IO Exception.
- 1. Checked Exception → Compile time exception
- 2. Unchecked Exception → Runtime exception.
(Ex: 1. Arithmetic Exception
2. InputMismatch Exception
3. Null Point Exception.)
- * Parent class of all Exceptions



Throwable → Exception → Runtime Exception → Arithmetic Exception.

- * Exo
- * Error → Hardware → Virtual Machine, Drivers, OS.
- * Exception → Software → Programmes.
- ↳ we can handle exceptions.

* Exception Handling:

1. try
2. catch
3. finally
4. throw
5. throws.

- * we can handle exception with try & catch.

try: write all your error prone code inside your try block.

Catch: * [Whenever an exception is encountered in try block & entered into immediately & exists from the try block & entered into catch block].
Used to handle the exception.

- * we cannot write try block only. we can write try with catch only.

```
int num3 = 0;
try {
```

```
    num3 = num1 / num2;
}
```

```
catch (Exception e) {
```

```
    // sysout ("Divided by zero is not possible");
    sysout(e); → (java.lang.ArithmeticException: / by zero)
    sysout(e.getMessage()); → (/ by zero)
}
```

⇒ Gracefull shutdown.

output

Enter number 1 :

100

Enter number 2 :

0

Divided by zero is not possible.

- * try & catch stop the program to terminate, when we have exception.

- * We can write multiple catch blocks, for one try block.
- * Always catch block with (Exception e) should be in the last of your code., if this catch block is in the first, the code will never entered to other catch blocks, so we get compile time exception.

 * `catch (ArithmeticException | InputMismatchException e) {`
 Index ^{↳ pipe symbol.}
 `| ArrayIndexOutOfBoundsException)`
`}`

 * We can write Exceptions with multiple other exceptions in one catch block.

* We can write multiple try-catch loops and we can write try-catch block inside try & catch blocks also.

```

try {
    try {
        }
        catch (Exception e) {
            try {
                }
            }
            catch (Exception e) {
                }
            }
        }
    }

```

 * We cannot write try without catch and vice versa.

finally:

finally:

- * This executes always irrespective of error/exception.
- * It doesn't depend on exception.
- * It is used ~~for~~ for cleanup purpose & resource closing.
- * In terms of Java, Scanner is called resource.
- * Resource block should not be write in try block.
- * Data base connections also a resource.

ex: try {

num3 = num1/num2;

}

catch (Exception e) {

}

finally {

sc.close();

}

[Definitely code will enter into finally block. So, resource closing always close in finally block].

* finally cannot execute when,

1. Force shutdown of your JVM - having method called

System.exit(0); → Random Number.

2. Stack Overflow - when we give an infinite loop i.e,

while(true) {

→ Out of memory error.

}

* We can use, 1. try - catch - finally ✓

2. try - catch - catch - finally. ✓

3. try - finally ✓

* We cannot use finally without try & catch.

4. try - catch - finally - finally ✗

* We cannot use multiple finally blocks.

- * finally is not mandatory to use in code.
- * Again we can write try-catch blocks in finally block.
- * Throws:
- * It is not used to handle the exception.
- * It is used at the method declaration.
- *** we can only delegate/cascade the exception to calling method.

Ex:

```

package com;
import java.io. File;
public class FileHandling {
    psvm ( ) throws FileNotFoundException {
        File file = new File("D:\\FLM - April 2025\\Hi!.txt");
        FileInputStream fileInputStream = new FileInputStream(file);
    }
}

psvm ( ) {
    FileHandling obj = new FileHandling();
    try {
        obj.openFile();
    }
    catch (Exception e) {
        sysout(e)
    }
}

public void openFile() throws Exception {
    File file = new File("D:\\FLM - April 2025\\Hi!.txt");
    FileInputStream fileInputStream = new FileInputStream(file);
}

```

Throw: Just used to throw an exception.

* Custom Exceptions

→ Real time exceptions (own exceptions).

* throw will not handle the exception, it will delegate the exception only.

* To handle this exception, we have to write in try-catch block only.

Ex: package com;

```
public class Test {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            syso("Enter age: ");
```

```
            sc = new Scanner(System.in);
```

```
            int age = sc.nextInt();
```

```
            if (age > 18) {
```

```
                syso("You can vote");
```

```
            }
```

```
            else {
```

```
                // syso("Not eligible to vote");
```

```
                throw new InvalidAgeException("You are not eligible");
```

```
            }
```

```
        catch (Exception e) {
```

```
            syso(e);
```

```
        }
```

Output: Enter age:

Not eligible to vote

Custom exception creation:

```
package com;
```

```
public class InvalidAgeException extends Exception {
```

```
    public InvalidAgeException(String message) {
```

```
        super(Exception);
```

```
        message
```

```
    }
```

Output: Enter age:

5

com.InvalidAgeException:

You are not eligible.

* There is only one way to handle exception in Java:

```
try {  
    }  
catch {  
    }  
}
```

* to create custom Exception:

- Create a class
- Extend with Exception / Runtime Exception
- Create a parameterized constructor.
- Call parent constructor using super()

* Global Exception Handler: (Spring Boot).

When we throw a custom Exception it will directly go to Global Exception Handler.
→ @ front end.

* ~~throw~~ throws used in method declaration, throw keyword used in catch blocks.

Ex: package com;

```
public class Employee {  
    private int empId;  
    private String name;  
    private double salary;
```

```
    public Employee () {
```

```
    }
```

```
    public Employee (int empId, String name, double salary) {  
        super();
```

```
        this empId = empId;
```

```
        this name = name;
```

```
        this salary = salary;
```

```
    }
```

```
public void setEmpId ( ) {
```

```
}
```

```
public int getEmpId ( ) {
```

```
    return empId;
```

```
}
```

```
public void setName ( ) {
```

```
}
```

```
public int getName ( ) {
```

```
    return Name;
```

```
}
```

```
public void setSalary ( ) {
```

```
}
```

```
public int getSalary ( ) {
```

```
    return salary;
```

```
}
```

```
public String toString ( ) {
```

```
    return
```

```
}
```

```
package com;
```

```
public class Company {
```

```
    String name;
```

```
    Employee [ ] employees; [to store multiple no. of Employees  
                             since, company has mbe no. of Employees]
```

```
    public void main ( ) {
```

```
        Employee emp1 = new Employee ( 1, "Gayatri", 100 );
```

```
        Employee emp2 = new Employee ( 2, "Ravi", 10000 );
```

```
        Employee emp3 = new Employee ( 3, "Sri", 45678 );
```



```

Employee[] emps = new Employee[3];
Company company = new Company();
company.name = "FLM";
company.employees = emps;
Syso(company);

```

Output:

Output:

```

Company [ name = FLM ; Employee [ empId = 1 ; name = Gayatri, salary = 100.0 ]
Employee [ empId = 2 ; name = Ravi, salary = 10000 ]
employee [ empId = 3 ; name = Sri, salary = 456 ]
Company company2 = new Company( "Facebook", emps );
Syso( company );

```

23/05/2025

Class: 28

File Handling.

* This can be done with help of,

1. Byte Streams

- File Input Stream
- File Output Stream.

* File is again a class in Java.

2. Character Streams

- File Reader
- Buffer Reader.

File Input Stream:

* This is used to read a file, it will ^{not} create a new file.

File Output Stream:

* This is used to write a file, it will create a new file.

Ex: package com; To read a file throws IOException

```
public class ByteStreams {  
    public static void main(String[] args) {  
        File file = new File("D:\\FLM-April 2015\\H1.txt");  
        FileInputStream fis = new FileInputStream(file);  
        while (fis.read() != -1) {  
            // reads a byte of data from this input stream.  
            // returns the next byte of data or -1 if the file is end of stream. (is reached).  
            int temp;  
            while (temp = fis.read() != -1) {  
                System.out.print(temp);  
                System.out.print((char) temp);  
            }  
        }  
    }  
}
```

fis.read();
↓
method to read the file.

```
catch (FileNotFoundException e) {  
    System.out.println(e);  
} finally {  
    try {  
        fis.close();  
    } catch (IOException e) {  
        System.out.println(e);  
    }  
}
```

Output:
E L L O 3 2 8 7 1 0 1 0 8 9 9 1 1 1 0 9 1 0 1 3 2 1 1 6 1 1 1 3 2 1 0 9 1 2 1 3 2 ...
Hello welcome to my Java class.

```
finally {  
    try {  
        fis.close();  
    }  
    catch (IOException e) {  
        System.out.println(e);  
    }  
}
```

* Automatically closes all the resources. with,

"try with resources block".

```
try (FileInputStream fileInputStream = new FileInputStream(file)) {  
    // Implementation written inside this block.  
}
```

* InputStream implements Closeable { }

to write a file.

```
public void writeIntoFile() {
    File file2 = new File("D:/FLM - April 2015 / Output.txt");
    File file = new File("D:/FLM - April 2015 / Hi.txt");
    FileOutputStream fos = null;

    try {
        fos = new FileOutputStream(file2);
        int temp;
        String data = "Hardcoded data";
        while (temp = fos.write(fos.write
        fos.write(data.getBytes()));
        while ((temp = fis.read()) != -1) {
            fos.write(temp);
        }
    } catch { (same as before code)
    }

    Psvm {
        ByteStreams byteStreams = new ByteStreams();
        byteStreams.writeIntoFile();
    }
}
```

(select all &
Ctrl + I → create &
put into new
method)

⇒ Hard coded Data Hello welcome to my Java class.

* Automatically closes all resources.

```
try (FileInputStream fileInputStream = new FileInputStream(file)) {
```

```
    Scanner sc = new Scanner(System.in)
```

```
    ) {
```

```
        int a = 10/0;
        sc.next();
```

```
    }
```

```
    catch (ArithmeticException e) {
```

```
        sysout(e)
```

```
    }
```

[while we using
try with resource block
we need not to use
finally block, due to
try block closes all
resources automatically]